The Relay — write-up

Numbers: `python3 scripts/tables.py results/results.csv`. Decisions and costs: `DECISIONS.md`. Eval questions were drafted with the assistant before any retrieval run (`author: placeholder`); graded rows read by hand in `results/eval_review.md`.

1. What I built and why

Go pipeline: **rewrite** (rules) → **retrieve** (own BM25) → **decide** (refuse with no model call if top normalised score < T) → **draft** (one Groq call) → **verify** (string checks, ≤1 redraft), behind a token-bucket limiter, a retry policy, one FIFO queue and a ladder: L0 full, L1 fewer chunks, L2 "busy, try in N s", L3 cited passages, no model call. **Deciding choice: under pressure give less, never something wrong.** Replies are held until every sentence cites a retrieved chunk and every quote is in its chunk word for word; uncited sentences are deleted. Cost: the first token *is* the whole answer.

2. The real service and my stand-in

Limit	Value found	How found	Confidence
Requests per minute	30	429 body; yet 34 of a 35-call burst passed	Value high; enforcement low
Tokens per minute	8,000	header; 8 concurrent 1.28K-token calls: 6 ok, 2 refused	High
Requests per day	1,000	header; reset grows 86.4 s per call (refill, not midnight)	High
Tokens per day	200,000	docs only	Low: not measured
Requests in flight	no cap up to 35	35 concurrent calls	Low
When a limit is hit	429 in 62–237 ms; `retry-after`; body names limit and wait	captured	High
Time to recover	RPM 2 s; TPM shortfall ÷ 133 tokens/s	poll; arithmetic on 429	Medium

From 61 planned calls (`results/probe/`). The **stand-in** speaks Groq's wire format, resamples probe latencies, adds 24 ms per request in flight, and refuses with Groq-shaped buckets and 429s. **Least like Groq:** real slowness comes from other tenants, not my load; its limits are exact where Groq leaked; its failures are chosen.

3. Results

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
Steady	real	16	268	3.6	2171 / 12274 / 12274	2171 / 12274 / 12274
Burst (20 at once)	stand-in	32	199	9.7	15 / 7479 / 7605	15 / 7479 / 7605
Sustained overload	stand-in	48	221	13.0	523 / 15175 / 19286	523 / 17585 / 20001
Per-minute limit halved	stand-in	12	199	3.6	450 / 9530 / 9530	450 / 9530 / 9530
Daily budget exhausted	stand-in	7	148	2.8	18 / 547 / 547	18 / 547 / 547

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers (list price)
Steady (real)	81.2	0.0	6.2	0.44	0	2466	$0.218
Burst	56.2	43.8	0.0	0.00	2	1396	$0.115
Sustained overload	70.8	27.1	2.1	0.04	3	1193	$0.099
Limit halved	91.7	8.3	0.0	0.00	0	1424	$0.117
Daily budget exhausted	28.6	71.4	0.0	0.00	0	1380	$0.114

Cost: $0 free; shown at $0.075/$0.30 per M tokens. Real steady's other 12.5% are dropped connections; rate-limited includes L2/L3. Stand-in steady: p50 448 ms, 100% ok. **Quality** (held-out, 10): 40% correct and cited, 40% wrong, 20% correctly refused, 2,012 tokens/answer; by hand 6 correct, 1 wrong (`results/eval_review.md`). **Unguessable:** 34 of 35 passed a 30 RPM limit; three correct answers were refused over U+2011 in "Mamba-2".

4. Where the time goes

Real steady `steady-R1-071824-0005`: 12,274 ms = **11,868 waiting** in our queue for TPM + **400 working** in the model + 6 ours, no retries; all before the first token.

5. When a limit is hit

Per minute: our limiter holds requests; admission estimates the wait for a request *plus everyone queued ahead* and gives the best rung that finishes in the person's 20 s (L0, else L1, else L2 at once). Burst: 17 L0, 1 L1, 14 L2, no timeouts, no provider 429s. Dropped: only someone whose deadline passes while queued. **Per day:** new requests get L3; running ones finish.

Retrieval: quality versus cost

Setting (piece size, pieces, re-rank)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
R1: 150 words, 6, no (dev)	10	40.0	20.0	30.0	1999	12243 / 21768	12243 / 21768
R2: 300 words, 4, no (dev)	10	40.0	20.0	30.0	2438	16156 / 32763	16156 / 32763
R3: 300 words, 2, no (dev)	10	20.0	30.0	30.0	1785	9311 / 24583	9311 / 24583
R1 (held-out, chosen)	10	40.0	40.0	20.0	2012	9095 / 25718	9095 / 25718

R1: R2's quality, 18% fewer tokens; tokens per minute binds. Near the limit L1 uses 2 chunks; at L3 retrieval is the answer. T = 0.13 only saves tokens: the model refused every unanswerable dev question.

How this library will mislead a naive retriever

(1) **Shared vocabulary:** "H-Net's latency in real-time spoken dialogue?" pulled Moshi chunks; the model answered "160ms" citing Moshi, and verify passed the real quote. (2) **Table-only facts:** "Parameters of the big Transformer?": 213M sits in Table 3, flattened to unlabelled numbers; the model refused. (3) **Page furniture:** Moshi's title on every page makes "What is Moshi?" match every chunk; we strip repeated lines.

6. Retries

Only 429, 5xx, timeouts, dropped connections; ≤2; the provider's hint plus jitter, else full-jitter backoff; never past the deadline. **Worse:** with limits silently halved, naive retries (fixed 1 s, hint ignored) hit 3 429s instead of 1 on `…-0011` and still failed, 11.8 s vs 9.5 s. Small: under plain overload both tied, as our limiter kept calls off Groq's 429s.

7. What the person sees

In ~20 ms, zero tokens: *"Our daily model budget is used up, so I can't write an answer. These passages from the papers best match your question:"* plus three cited excerpts: never wrong, maybe not an answer.

8. What I'm unsure about, what I left out, what I learned

Least certain: 20 s patience; at 5 s most queued answers under load are lost and the ladder must step down earlier. **Not built:** a re-ranker; a check that a cited paper is the one the question names (would have refused q19). **Hardest:** fast, good, cheap and nobody turned away cannot all win, so I chose an order: correct, then available (always *some* reply), then cheap, then fast. The ladder is that order made explicit. **New to me: rate limits are buckets that refill, not counters that reset.** I expected "1,000 requests per day" to reset at midnight. The probe showed `x-ratelimit-reset-requests` growing by 86.4 s with every call: 86,400 s ÷ 1,000. The allowance refills continuously, so "daily budget exhausted" is not a cliff: one request comes back every 86.4 s, and both the limiter and the stand-in had to be built around that.

Your feedback on Vaani

I have followed Vaani for about a year, since I first reached out to Tushar, whose work inspired me. Going through the docs after launch, the range of features stood out, especially interruption handling (new to me) and ambient background sound. Confusing: two docs domains (docs.vaani.dev, docs.vaanivoice.ai), and docs that open on a REST API page with no overview of how a call flows. I would change: the Quickstart has only cURL and Python; SDKs or samples in Node, Go and Java would meet more teams where they are. I built this in Go and wrote my client by hand.

How I worked with the assistant

I started by sitting with Claude to understand what we were building, then did the system design myself and wrote `DESIGN.md`, where every decision was taken after careful consideration. With the build plan ready, building it was a matter of time. I hold one rule: never ship a line of code I don't understand. AI is good when it raises your speed and efficiency, not when it makes your decisions, and that is where I spent most of my effort in this project: the design and architecture. Most of the code was produced by Claude, but I went through every file, and from the very beginning every file and function has its own tests; each new feature ran the full suite (`gofmt`, `go vet`, tests with the race detector) before its commit, so we never broke something or faked it.

Prompts that mattered: giving it my design with one rule, *give less, never something wrong*; "run the probe", which turned assumed limits into measured ones; and "fix and rerun", which found three more bugs instead of shipping the first numbers. **Caught:** verify let an uncited sentence from the model's own knowledge through; all tests passed, and I caught it on the first real Groq answer. Later its first fix for a "busy instead of passages" bug was not the cause; a test reproducing the run found the real one. **Overruled:** it recommended writing my own questions before any eval; I chose to run with its drafted questions to keep moving; the cost is that my quality numbers rest on questions I reviewed rather than wrote. **Distinctly mine:** the one-rule design, a single model call with rule-based rewriting, the zero-token L3 rung, the open-loop generator with its lag check, and measuring limits instead of trusting the docs.